

Guía de laboratorio — Construya su primer pipeline de CI/CD

Tecnológico de Costa Rica · Escuela de Ingeniería Electrónica · Taller de Sistemas Embebidos
Mini-taller: Integración y entrega continua de software embebido · Caso Jenkins

Todos los comandos de esta guía están verificados sobre WSL2 con Ubuntu 24.04, Docker Engine 29 y Jenkins LTS. El trabajo se realiza **en parejas**: cada integrante ejecuta su propio Jenkins, pero ambos publican al mismo repositorio. Esa es la situación que la integración continua existe para resolver.

Antes de empezar

Convenciones

\$ lo escribe usted en la terminal. No copie el símbolo.
comentario explicativo. No lo escriba.
> salida esperada del comando.

Todo el trabajo vive en `~/validador-ci`. Los integrantes de la pareja se identifican como **A** y **B**; decidan ahora quién es quién.

Qué necesita su máquina

Requisito	Detalle
Sistema	WSL2 con Ubuntu 22.04 o superior, o Linux nativo. También sirve macOS con Docker Desktop
Docker	Docker Engine 24 o superior con Compose v2. Verifique con <code>docker compose version</code>
Git	Cualquier versión reciente
Cuenta de GitHub	Con un token de acceso personal tipo <i>classic</i> y alcance repo ya generado
Puerto 8080	Libre. Si tiene otro Jenkins corriendo, deténgalo
Disco	3 GB libres para la imagen del contenedor

ADVERTENCIA — GENERE EL TOKEN ANTES DE LA SESIÓN

GitHub dejó de aceptar contraseñas para operaciones de Git en 2021. Necesita un token de acceso personal, y generarlo en clase consume varios minutos por persona.

Vaya a *Settings* → *Developer settings* → *Personal access tokens* → **Tokens (classic)** → *Generate new token*, marque el alcance **repo** y guárdelo: solo se muestra una vez.

Los tokens *fine-grained* también sirven, pero se otorgan a repositorios específicos; como el repositorio de la pareja aún no existe, tendría que volver a editarlos después. Use *classic*.

Verificación rápida

```
docker --version      # 24 o superior
docker compose version # v2
git --version
ss -tlnp | grep 8080   # no debe devolver nada
df -h $HOME            # al menos 3 GB libres
```

0 Descargue el material y levante Jenkins

10 min

El material del taller está en el repositorio del expositor. Descargue el `.zip` desde *Code* → *Download ZIP* y quédese solo con la carpeta **taller-estudiantes**.

```
mkdir -p ~/validador-ci
# copie aquí el contenido de taller-estudiantes/
cd ~/validador-ci
ls
# > docker Jenkinsfile Makefile src tests
```

Construya la imagen y levante el contenedor:

```
cd ~/validador-ci/docker
DOCKER_BUILDKIT=0 docker build -t jenkins-validador:v1 .
docker compose up -d --no-build
docker compose ps
```

NOTA — POR QUÉ EL DOCKERFILE EXISTE

La imagen oficial de Jenkins incluye Java y el servidor, pero **no incluye compilador de C**. Si el pipeline va a ejecutar **make**, el entorno de construcción tiene que existir dentro de la imagen. Abra **docker/Dockerfile** y véalo: es la primera idea del taller.

El **docker-compose.yml** persiste **jenkins_home** en un volumen nombrado. Si detiene el contenedor no pierde nada.

Obtenga la clave de desbloqueo y entre a <http://localhost:8080>:

```
docker exec jenkins-validador \
  cat /var/jenkins_home/secrets/initialAdminPassword
```

En el asistente elija *Select plugins to install* y luego **None**: los complementos vienen precargados desde el **Dockerfile** y no hace falta descargar nada. Cree su usuario administrador.

ADVERTENCIA — ERROR CONOCIDO EN WSL2

Si **docker build** falla con **bind source path does not exist: /usr/lib/wsl**, es BuildKit intentando montar una ruta que no existe en su WSL. Por eso el comando lleva **DOCKER_BUILDKIT=0** delante: usa el constructor clásico y evita el problema. Si ya lo puso y aun así falla, avise.

1 Cree el repositorio de la pareja

8 min

ROLES EN ESTE PASO

A crea el repositorio y hace el primer envío. **B** espera la invitación como colaborador.

A crea en github.com/new un repositorio llamado **validador-credenciales**, **público**, y **sin README** ni **.gitignore**. Después:

```
cd ~/validador-ci
git init
git branch -M main
git config user.name "Su Nombre"
git config user.email "su-correo@ejemplo.com"
git add .
git status --short          # no debe aparecer build/
git commit -m "Material inicial del taller"
git remote add origin https://github.com/USUARIO-A/validador-credenciales.git
git push -u origin main
```

Cuando pida contraseña, pegue el **token**, no su contraseña de GitHub. La terminal no muestra nada mientras lo pega: es normal.

A agrega a **B** como colaborador en *Settings* → *Collaborators*. **B** acepta la invitación y clona:

```
cd ~
git clone https://github.com/USUARIO-A/validador-credenciales.git validador-ci
cd validador-ci
git config user.name "Su Nombre"
git config user.email "su-correo@ejemplo.com"
git config pull.rebase true
```

NOTA — EL REPOSITORIO CREADO VACÍO NO ES CAPRICHIO

Si crea el repositorio con README, el remoto tendrá un commit que usted no tiene y el primer **push** será rechazado con **[rejected] main ->main (fetch first)**. Se arregla con **git pull --rebase origin main**, pero es más simple no provocarlo.

2 Conecte Jenkins al repositorio

7 min

Ambos integrantes hacen este paso, cada uno en su propio Jenkins.

En `http://localhost:8080`: *Nueva Tarea* → nombre `validador-ci` → tipo **Pipeline** → *OK*. En la sección *Pipeline*:

Campo	Valor
Definition	Pipeline script from SCM
SCM	Git
Repository URL	la URL del repositorio de la pareja
Credentials	- none - (el repositorio es público)
Branch Specifier	**/*
Script Path	Jenkinsfile

Guarde y pulse **Construir ahora**.

ADVERTENCIA — EL COMODÍN ES **/*, NO **

Con ****** a secas el trabajo no encuentra ninguna rama y el build sale en rojo. Necesitamos el comodín porque en el paso 4 vamos a trabajar con ramas y Jenkins debe construirlas todas, no solo `main`.

Qué debe ver

El build queda **AMARILLO** (inestable), no rojo ni verde:

```
> OK: binario en build/bin/validador-credenciales (version 0.1.1)
> [ FALLA ] test_formato_acepta_ocho_digitos
> Resumen: 1 pruebas, 1 fallas
> Setting overall build result to UNSTABLE
> Stage "Empaquetar" skipped due to when conditional
> Finished: UNSTABLE
```

NOTA — AMARILLO NO ES LO MISMO QUE ROJO

Rojo (FAILURE): el código no compiló. Está roto.

Amarillo (UNSTABLE): compiló, pero el comportamiento no es el esperado. Una prueba falló.

Su primer build es amarillo a propósito: el código que recibió es un esqueleto que compila pero no hace nada. Su trabajo empieza aquí.

Note además que las etapas *Empaquetar* y *Entregar* se omitieron. Nadie lo decidió: lo decidió la condición **when** del `Jenkinsfile`. No se empaqueta código que no pasó las pruebas.

3 Ronda 1 — trabajo en ramas separadas

18 min

ROLES EN ESTE PASO

A implementa `cred_formato_valido` y `cred_checksum` en `src/credencial.c`.

B escribe las pruebas de esas dos funciones en `tests/test_credencial.c`.

No se muestren el código. Ambos trabajan contra la especificación del Anexo A. Si la interpretan distinto, el pipeline lo va a detectar, y eso es precisamente lo que queremos observar.

Cada uno crea su rama:

```
# A
git checkout -b implementacion-formato

# B
git checkout -b pruebas-formato
```

Trabajen, verifiquen localmente y publiquen:

```
make clean && make test          # antes de publicar, siempre
git add -A
git commit -m "describe que hizo"
git push -u origin NOMBRE-DE-SU-RAMA
```

NOTA — POR QUÉ SU RAMA NO GENERA PAQUETE

Al publicar, su Jenkins construye la rama: compila y prueba. Pero las etapas *Empaquetar* y *Entregar* se omiten aunque el build salga verde.

Es deliberado. Un artefacto entregable solo se produce a partir de código ya integrado en **main** y revisado por alguien más. Su rama es trabajo en curso.

Punto de verificación

- ☐ Ambas ramas existen en GitHub.
- ☐ Ambos builds aparecen en Jenkins con el nombre de la rama en la etapa *Preparar*.
- ☐ Ninguno de los dos produjo artefacto.

4 Pull request y revisión cruzada

10 min

En GitHub, cada uno abre un *pull request* de su rama hacia **main**. En la descripción, expliquen en una o dos líneas qué hicieron.

Después **intercambien**: cada uno revisa el PR del otro. Lea el código del compañero y pregunte lo que no entienda. Cuando esté conforme, pulse *Merge pull request*.

Fusionen primero el de **A** (la implementación) y después el de **B** (las pruebas).

ADVERTENCIA — ES PROBABLE QUE EL BUILD DE `main` SALGA AMARILLO

Si A y B interpretaron la especificación de forma distinta, al integrarse ambos cambios las pruebas de B van a fallar contra la implementación de A.

Eso no es un error suyo: es el hallazgo del taller. Dos personas trabajaron en aislamiento y sus supuestos no coincidían. La integración continua lo detectó en minutos, no en semanas.

Acuerden la interpretación correcta consultando el Anexo A, corrijan quien deba corregir, y vuelvan a publicar.

Actualicen su copia local:

```
git checkout main
git pull
```

Punto de verificación

- ☐ **main** está en verde.
- ☐ El build de **main** **sí** produjo artefacto: búsquelo en *Last Successful Artifacts*, en la página del proyecto (no en la de cada ejecución).

5 El choque — conflicto de fusión

8 min

Este paso provoca un conflicto a propósito. Háganlo exactamente como se indica.

Ambos, **al mismo tiempo**, crean una rama y modifican la *misma* línea de `src/credencial.h`:

```
# A
git checkout main && git pull
git checkout -b uid-largo
# edite credencial.h: CRED_LONGITUD pasa de 8u a 10u
# ajuste tambien las credenciales de ejemplo en src/main.c
make clean && make test
git commit -am "Amplia el UID a 10 digitos"
git push -u origin uid-largo

# B
git checkout main && git pull
git checkout -b uid-corto
# edite credencial.h: CRED_LONGITUD pasa de 8u a 6u
# ajuste tambien las credenciales de ejemplo en src/main.c
make clean && make test
git commit -am "Reduce el UID a 6 digitos"
git push -u origin uid-corto
```

Abren los dos pull requests. Fusionen el primero: entra sin problema. Al intentar el segundo, GitHub muestra *This branch has conflicts that must be resolved*.

Resuelvan el conflicto: decidan entre los dos qué longitud queda, éditenlo, y completen la fusión.

NOTA — QUÉ ACABA DE PASAR

Git no puede decidir cuál de las dos versiones es correcta, porque ambas son sintácticamente válidas. Solo ustedes saben cuál refleja el requisito.

En un equipo real esto ocurre a diario. Cuanto más tiempo pasa una rama sin integrarse, más grande y más difícil es el conflicto. De ahí el nombre: integración *continua*.

6 Ronda 2 — roles invertidos

15 min

ROLES EN ESTE PASO

B implementa `cred_autorizar`.

A escribe las pruebas de `cred_autorizar`.

Misma mecánica: rama propia, pull request, revisión cruzada.

`cred_autorizar` combina las dos funciones anteriores con una lista de credenciales permitidas. Su especificación está en el Anexo A, e incluye un detalle que conviene leer con atención: el **orden** en que se aplican las verificaciones.

NOTA — EL ORDEN IMPORTA

Si una credencial tiene mal formato *y* no está en la lista de permitidas, ¿qué error devuelve?

La especificación lo define. Si A y B no leen lo mismo, las pruebas van a fallar. Otra vez: mejor descubrirlo ahora que en producción.

7 Modifique el pipeline

10 min

Hasta ahora usaron un pipeline que les vino dado. Ahora lo van a modificar.

El `Makefile` ya tiene un objetivo `analyze` que ejecuta `cppcheck`, un analizador estático. Pruébenlo:

```
make analyze
```

Agreguen una etapa al `Jenkinsfile` que lo invoque, entre *Compilar* y *Probar*:

```
stage("Analizar") {
    steps {
        sh "make analyze"
    }
}
```

Publíquenlo por el mismo flujo: rama, pull request, revisión, fusión.

NOTA — ESTO ES PIPELINE AS CODE

Acaban de cambiar el proceso de construcción con un *commit*. Quedó revisado por su compañero, registrado en el historial y es reversible.

Si el pipeline estuviera configurado desde formularios de la interfaz web, ese cambio no tendría autor, ni fecha, ni revisión, ni forma de deshacerse.

Desafío opcional: haga que la etapa *Analizar* solo se ejecute en `main`, usando una condición `when` como la de *Empaquetar*.

8 Rómpalo a propósito

6 min

Con `main` en verde y todas las pruebas pasando, provoquen un fallo deliberado.

```
git checkout main && git pull
git checkout -b experimento-roto
# edite src/credencial.h y cambie CRED_LONGITUD a un valor distinto
git commit -am "Experimento: cambia la longitud del UID"
git push -u origin experimento-roto
```

Observen el build de esa rama en Jenkins. Anoten:

Pregunta	Su respuesta
¿El build quedó rojo o amarillo?	
¿Cuántas pruebas fallaron?	
¿Se generó artefacto?	
¿Cuánto tardó en enterarse?	

Después, borren la rama sin fusionarla:

```
git checkout main
git push origin --delete experimento-roto
git branch -D experimento-roto
```

NOTA — LA RED DE SEGURIDAD

Un cambio de un carácter, sintácticamente válido, alteró el comportamiento de un sistema de control de acceso. Las pruebas lo detectaron y el pipeline impidió que se produjera un artefacto entregable.

En software embebido eso importa más que en la web: un firmware defectuoso puede llegar a miles de dispositivos que nadie va a actualizar en años.

Lista de verificación de salida

- ☐ Levanté Jenkins en un contenedor y entiendo por qué el `Dockerfile` instala un compilador.
- ☐ Conecté Jenkins a un repositorio de GitHub mediante *Pipeline script from SCM*.
- ☐ Vi un build amarillo y sé en qué se diferencia de uno rojo.
- ☐ Trabajé en una rama propia y abrí un *pull request*.
- ☐ Revisé el código de mi compañero antes de fusionarlo.
- ☐ Comprobé que una rama de trabajo no genera artefacto y `main` sí.
- ☐ Provoqué un conflicto de fusión y lo resolví de acuerdo con mi compañero.
- ☐ Implementé una función y escribí pruebas para otra.
- ☐ Agregué una etapa al `Jenkinsfile` y la publiqué por *pull request*.
- ☐ Rompí el build a propósito y observé la reacción del pipeline.
- ☐ No borré `~/validador-ci`: es el punto de partida de la próxima sesión.

Anexo A — Especificación de las funciones

Esta es la fuente de verdad. Ante cualquier duda sobre qué debe hacer una función, gana lo que diga este anexo.

Constantes

```
#define CRED_LONGITUD 8u    /* dígitos que debe tener un UID */
#define CRED_MAX_PERMIT 16u /* tope de la lista de permitidas */
```

`int cred_formato_valido(const char *uid)`

Verifica que la credencial tenga exactamente `CRED_LONGITUD` caracteres y que **todos** sean dígitos decimales ('0' a '9').

Entrada	Retorno
"12345678"	1
"1234567"	0 (muy corta)
"123456789"	0 (muy larga)
"1234567A"	0 (contiene una letra)
	0
NULL	0

`int cred_checksum(const char *uid)`

Calcula el dígito verificador de los primeros `CRED_LONGITUD - 1` caracteres: suma sus valores numéricos y devuelve esa suma **módulo 10**.

Ejemplo detallado con "12345678": se toman los primeros siete dígitos, $1 + 2 + 3 + 4 + 5 + 6 + 7 = 28$, y $28 \bmod 10 = 8$.

Entrada	Retorno
"12345678"	8 (suma 28)
"11111115"	6 (suma 6)
"1234567A"	-1 (formato inválido)
NULL	-1

El octavo carácter **no** participa en la suma: es el dígito que se está verificando.

`cred_resultado_t cred_autorizar(...)`

```
cred_resultado_t cred_autorizar(const char *uid,
                                const char *permitidas[],
                                size_t n);
```

Decide si una credencial concede acceso. Aplica tres verificaciones **en este orden**, y devuelve el error de la primera que falle:

#	Verificación	Si falla, devuelve
1	Formato válido	CRED_ERR_FORMATO
2	El último dígito coincide con el checksum calculado	CRED_ERR_CHECKSUM
3	El uid aparece en <code>permitidas</code>	CRED_ERR_NO_AUTORIZADA

Si las tres se cumplen, devuelve `CRED_OK`.

NOTA — CASOS QUE DEBE CONTEMPLAR

`uid == NULL` → `CRED_ERR_FORMATO`.

`permitidas == NULL` o `n == 0` → ninguna credencial está autorizada, de modo que una credencial bien formada y con checksum correcto debe devolver `CRED_ERR_NO_AUTORIZADA`, no `CRED_OK`.

Una credencial con formato inválido *y* ausente de la lista devuelve `CRED_ERR_FORMATO`, porque esa verificación va primero.

Anexo B — Cómo escribir una prueba

El archivo `tests/test_credencial.c` ya trae una prueba de ejemplo. Use ese patrón:

```
static void test_formato_rechaza_letra(void)
{
    MT_ASSERT_EQ(cred_formato_valido("1234567A"), 0);
}

int main(void)
{
    MT_PRUEBA(test_formato_acepta_ocho_digitos);
    MT_PRUEBA(test_formato_rechaza_letra); /* agregue aqui */

    return mt_informe("build/reports/credencial.xml", "credencial");
}
```

Macro	Uso
<code>MT_ASSERT(cond)</code>	Falla si la condición es falsa
<code>MT_ASSERT_EQ(obt, esp)</code>	Falla si los enteros difieren; informa ambos valores
<code>MT_PRUEBA(fn)</code>	Registra y ejecuta una función de prueba

NOTA — QUÉ HACE UNA BUENA PRUEBA

Una prueba por comportamiento, con nombre descriptivo. No escriba una sola función gigante que verifique todo: si falla, no sabrá qué se rompió.

Cubra los casos límite, no solo el camino feliz. Una entrada vacía, un `NULL`, un valor justo en el borde del rango. Ahí es donde viven los defectos.

Anexo C — Problemas frecuentes

Síntoma	Solución
<code>bind source path does not exist: /usr/lib/wsl</code>	Anteponga <code>DOCKER_BUILDKIT=0</code> al <code>docker build</code>
Password authentication is not supported	Use el token de acceso personal como contraseña, no la de su cuenta
Permission denied ... 403 con token válido	El token no tiene el alcance <code>repo</code> , o es <i>fine-grained</i> sin acceso a ese repositorio
[rejected] main ->main (fetch first)	<code>git pull --rebase origin main</code> y vuelva a publicar
Need to specify how to reconcile divergent branches	<code>git config pull.rebase true</code>
El build sale rojo y no encuentra el Makefile	El <i>Script Path</i> debe ser <code>Jenkinsfile</code> y el repositorio debe tener el <code>Makefile</code> en su raíz
Ninguna rama se construye	El <i>Branch Specifier</i> debe ser <code>**/*</code> , no <code>**</code>
No encuentro el artefacto	Está en la página del <i>proyecto</i> , bajo <i>Last Successful Artifacts</i> , no en la de cada ejecución
Lightweight checkout support not available	Es un aviso, no un error. Jenkins clona dos veces cuando el especificador usa comodines
El puerto 8080 está ocupado	<code>docker ps</code> para ver qué lo usa, y <code>docker stop NOMBRE</code>
Perdí la clave inicial de Jenkins	<code>docker exec jenkins-validador cat /var/jenkins_home/secrets/initialAdminPassword</code>

Guía elaborada para el mini-taller sobre integración y entrega continua de software embebido. Todos los comandos fueron verificados sobre el entorno descrito en los requisitos.